

- `TypeError: hello _name() missing 1 required positional argument: 'fname'`

signifying that you need to enter exactly 1 “required” argument for this function to work properly. Also, note the word “positional” in the error message. This refers to where the arguments are displayed. So in the function *greeting_user*, if you pass the greeting first (*how are you*) and username second (*sanket*), the output will make no sense at all – *Welcome how are you, sanket*. Hence the position also matters when you enter in the parameters of the argument.

Keyword arguments are where you define the argument when calling the function. Look at this code

- ```
def opening(line):
 "This prints a passed string into this function"
 print (line)
```

In this case, when you call the function, enter in the argument defining the parameter “line” given in the function name.

- ```
opening( line = "Welcome to Fast Track")
opening( line = "Enjoy this edition of Fast Track by
Digit")
```

If you just call `opening` without defining “line” you will see the error, `NameError: name 'line' is not defined`. An interesting note here – if you call the function without writing anything in the parentheses i.e. `opening()`, you will get the error- `TypeError: opening() missing 1 required positional argument: 'line'`.

Look at this example

- ```
def printinfo(name, age):
 "This prints a passed info into this function"
 print ("Name: ", name)
 print ("Age: ", age)
```

In this case, there are 2 arguments given in the function. When calling the function, you will have to define both the parameters- name and age. Since they are defined and every parameter will be printed in a separate line, the order of definition is immaterial. This is the key difference between the required argument and keyword argument when applied in this manner. So you can call the function as

- ```
printinfo( age = 17, name = "Digit" )
```

Note that age is given first while calling whereas its the second argument in the definition. But the output received will be the same